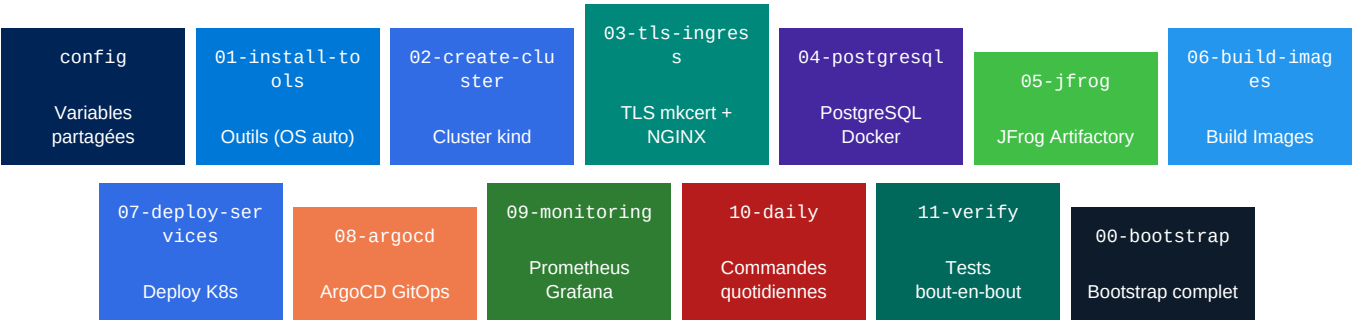


SCRIPTS POWERSHELL (.ps1)

ShipClass Local

13 scripts autonomes pour faire tourner tout le projet en local

Windows · macOS · Linux — PowerShell 7+ — Zéro cloud requis



Un seul script suffit pour tout installer : **./00-bootstrap.ps1**. Chaque script est indépendant, documenté, réexécutable et compatible Windows, macOS et Linux (PowerShell 7+ requis).

Architecture des 13 scripts PowerShell, leur ordre d'exécution, leurs paramètres et les dépendances entre eux.

Script	Paramètres clés	Dépend de	Durée ~	Idempotent ?
config.ps1	— (importé par tous)	—	0s	✓ Oui
01-install-tools.ps1	-Verify	—	5-15min	✓ Oui
02-create-cluster.ps1	-Reset	01	3min	✓ Oui
03-tls-ingress.ps1	—	02	3min	✓ Oui
04-postgresql.ps1	-Reset	02	1min	✓ Oui
05-jfrog.ps1	-Reset	02	3min	✓ Oui
06-build-images.ps1	-Service -Tag -Registry kind jfrog -ScanCVE	05	2min/svc	✓ Oui
07-deploy-services.ps1	-Service -Registry kind jfrog -Tag	03,04,06	1min	✓ Oui
08-argocd.ps1	-GitRepo -GitToken -InstallGitea	03	3min	✓ Oui
09-monitoring.ps1	—	03	4min	✓ Oui
10-daily.ps1	-Action start stop status rebuild db logs port-forward reset-all	Tous	0s	✓ Oui
11-verify.ps1	—	Tous	1min	✓ Oui
00-bootstrap.ps1	-SkipTools -SkipJFrog -GitRepo -GitToken	—	30-45min	✓ Oui

Prérequis	Windows	macOS	Linux
PowerShell 7+	winget install Microsoft.PowerShell	brew install powershell	apt install powershell
Docker Desktop	winget install Docker.DockerDesktop	brew install --cask docker	curl -fsSL https://get.docker.com sh
Droits Admin/sudo	Lancer PowerShell en admin	sudo disponible	sudo disponible
RAM disponible	Min 8 GB libres	Min 8 GB libres	Min 8 GB libres
Connexion internet	Pour télécharger les images Docker	Idem	Idem

[illegible]

```
./01-install-tools.ps1 -Verify # Vérifier les outils pwsh ./02-create-cluster.ps1 # Créer le cluster kind pwsh
./03-tls-ingress.ps1 # TLS + Ingress pwsh ./04-postgresql.ps1 # PostgreSQL pwsh ./05-jfrog.ps1 # JFrog Artifactory pwsh
./06-build-images.ps1 # Build toutes les images pwsh ./07-deploy-services.ps1 # Déployer les services pwsh
./08-argocd.ps1 # ArgoCD pwsh ./09-monitoring.ps1 # Prometheus + Grafana pwsh ./11-verify.ps1 # Tests finaux # ■■
Politique d'exécution Windows (si non déjà définie) ■■■■■■ Set-ExecutionPolicy RemoteSigned -Scope
CurrentUser
```

■ Sur Windows, exécuter PowerShell en tant qu'Administrateur pour les étapes 01 (winget), 03 (modification /etc/hosts) et 05 (modification daemon.json Docker).

PARTIE 2 — Scripts Détaillés

Code source complet de chaque script PowerShell avec explications. Chaque script est autonome, réexécutable et gère les cas d'erreur.

00.p
s1

config.ps1

Variables partagées et fonctions utilitaires

Ce script n'est pas exécuté directement. Il est importé par tous les autres via '. "\$PSScriptRoot/config.ps1"'. Il centralise toutes les variables (noms de conteneurs, mots de passe, domaines, namespaces) et définit les fonctions d'affichage Write-OK, Write-Fail, Write-Step, ainsi que Apply-Yaml qui applique un manifest YAML inline sans fichier temporaire permanent.

Paramètre / Élément	Description
\$Global:SC	Hashtable globale avec toutes les variables du projet
Write-Step/OK/Fail/Info	Fonctions d'affichage colorées pour un suivi clair
Apply-Yaml	Applique un manifest YAML depuis une string PowerShell
Assert-Command	Vérifie qu'un outil CLI est disponible dans le PATH

Code source du script — config.ps1

■ config.ps1

```
# config.ps1 – Variables globales partagées par tous les scripts ShipClass Local # Placer dans :
~/shipclass-local/config.ps1 # Importer dans chaque script avec : . "$PSScriptRoot/config.ps1"
$Global:SC = @{
  ClusterName = "shipclass-local"
  Namespace = "shipclass"
  Context = "kind-shipclass-local"
  Registry = "localhost:5001"
  RegistryName = "kind-registry"
  JFrogPort = 8082
  JFrogUser = "admin"
  JFrogPassword = "ShipLocal2024!"
  JFrogRepo = "docker-shipclass-local"
  PgContainer = "shipclass-postgres"
  PgPort = 5432
  PgUser = "pgadmin"
  PgPassword = "shipclass_local_pw"
  PgDatabases = @( "vessels_db", "classification_db", "certificates_db", "inspections_db", "documents_db" )
  Domains = @{
    Api = "api.shipclass.local"
    App = "shipclass.local"
    ArgoCD = "argocd.shipclass.local"
    Grafana = "grafana.shipclass.local"
    JFrog = "jfrog.shipclass.local"
    Gitea = "gitea.shipclass.local"
  }
  WorkDir = (Join-Path $HOME "shipclass-local")
  Services = @( "vessel-service", "classification-service", "certificate-service", "inspection-service", "document-service", "notification-service", "frontend" )
}

function Write-Step {
  param($msg)
  Write-Host "`n>> $msg" -ForegroundColor Cyan
}

function Write-OK {
  param($msg)
  Write-Host " [OK] $msg" -ForegroundColor Green
}

function Write-Fail {
  param($msg)
  Write-Host " [ERR] $msg" -ForegroundColor Red
}

function Write-Info {
  param($msg)
  Write-Host " [i] $msg" -ForegroundColor Yellow
}

function Write-Title {
  param($msg)
  Write-Host "`n$($"*62)`n"
  $msg`n$($"*62)" -ForegroundColor Magenta
}

function Assert-Command {
  param($Name)
  if (-not (Get-Command $Name -ErrorAction SilentlyContinue)) {
    Write-Fail "$Name n'est pas installé ou pas dans le PATH"; exit 1
  }
  Write-OK "$Name : $((& $Name --version 2>&1 | Select-Object -First 1) -replace "`n','")"
}

function Apply-Yaml {
  param($yaml)
  $tmp = [System.IO.Path]::GetTempFileName() + ".yaml"
  $yaml | Out-File $tmp -Encoding UTF8
  kubectl apply -f $tmp
  Remove-Item $tmp -Force
}
```

01-install-tools.ps1

01

Installation automatique des outils selon l'OS

Détecte automatiquement l'OS (\$IsWindows, \$IsMacOS, sinon Linux) et installe les outils appropriés. Sur Windows : winget + Scoop. Sur macOS : Homebrew. Sur Linux : curl + apt. Le paramètre -Verify permet de vérifier que tout est installé sans réinstaller.

Paramètre / Élément	Description
-Verify	Vérifier les installations sans réinstaller
winget (Windows)	Gestionnaire de paquets Windows 10/11 intégré
Scoop (Windows)	Pour kind et mkcert non disponibles dans winget
brew (macOS)	Homebrew avec brew install pour tous les outils
curl + apt (Linux)	Installation directe des binaires depuis les releases GitHub

Code source du script — 01-install-tools.ps1

■ 01-install-tools.ps1

```
#!/usr/bin/env pwsh # 01-install-tools.ps1 # Installe tous les outils requis selon l'OS detecte # Windows : executer en
tant qu'Administrateur param([switch]$Verify) . "$PSScriptRoot/config.ps1" Write-Title "ETAPE 01 - Installation des
outils" $os = if ($IsWindows) { "windows" } elseif ($IsMacOS) { "macos" } else { "linux" } Write-Info "OS detecte : $os"
if ($Verify) { Write-Step "Verification des outils" @"(docker","kubect","helm","kind","argocd","mkcert","git","jq") |
ForEach-Object { if (Get-Command $_ -ErrorAction SilentlyContinue) { Write-OK "$_ disponible" } else { Write-Fail "$_ :
NON INSTALLE" } } return } # ===== # WINDOWS #
===== if ($IsWindows) { Write-Step "Installation via winget" @(
"Docker.DockerDesktop", "Kubernetes.kubect", "Helm.Helm", "Git.Git", "jqlang.jq", "Microsoft.PowerShell" ) |
ForEach-Object { Write-Info "winget install $_" winget install --id $_ --silent --accept-package-agreements
--accept-source-agreements 2>&1 | Out-Null Write-OK " $_ installe" } Write-Step "Installation Scoop (kind + mkcert)" if
(-not (Get-Command scoop -ErrorAction SilentlyContinue)) { Set-ExecutionPolicy RemoteSigned -Scope CurrentUser -Force
Invoke-RestMethod https://get.scoop.sh | Invoke-Expression } scoop install kind mkcert Write-Step "Installation ArgoCD
CLI" $v = (Invoke-RestMethod https://api.github.com/repos/argoproj/argo-cd/releases/latest).tag_name Invoke-WebRequest
"https://github.com/argoproj/argo-cd/releases/download/$v/argocd-windows-amd64.exe" -OutFile
"$env:LOCALAPPDATA\Microsoft\WindowsApps\argocd.exe" -UseBasicParsing Write-OK "ArgoCD CLI installe" Write-Step
"Activation WSL2 (requis pour Docker Desktop)" wsl --install --no-distribution 2>&1 | Out-Null Write-Info "Un redemarrage
peut etre necessaire apres WSL2" } # ===== # MACOS #
===== elseif ($IsMacOS) { Write-Step "Installation via Homebrew" if
(-not (Get-Command brew -ErrorAction SilentlyContinue)) { /bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)" }
@"(kubect","helm","kind","git","jq","mkcert","argocd","derailed/k9s/k9s") | ForEach-Object { Write-Info "brew install
$_" brew install $_ 2>&1 | Select-Object -Last 1 } Write-Step "Docker Desktop macOS" brew install --cask docker Write-Info
"Lancer Docker Desktop depuis les Applications" } # ===== # LINUX #
===== else { Write-Step "Installation sur Linux (Ubuntu/Debian)" bash
-c "curl -fsSL https://get.docker.com | sh" sudo usermod -aG docker $env:USER $k8sVer = (Invoke-RestMethod
https://dl.k8s.io/release/stable.txt).Trim() Invoke-WebRequest
"https://dl.k8s.io/release/$k8sVer/bin/linux/amd64/kubect" -OutFile /tmp/kubect sudo chmod +x /tmp/kubect; sudo mv
/tmp/kubect /usr/local/bin/ bash -c "curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash"
Invoke-WebRequest "https://kind.sigs.k8s.io/dl/v0.23.0/kind-linux-amd64" -OutFile /tmp/kind sudo chmod +x /tmp/kind; sudo
mv /tmp/kind /usr/local/bin/ sudo apt-get install -y libnss3-tools -qq Invoke-WebRequest
"https://github.com/FiloSottile/mkcert/releases/download/v1.4.4/mkcert-v1.4.4-linux-amd64" -OutFile /tmp/mkcert sudo
chmod +x /tmp/mkcert; sudo mv /tmp/mkcert /usr/local/bin/ Invoke-WebRequest
"https://github.com/argoproj/argo-cd/releases/latest/download/argocd-linux-amd64" -OutFile /tmp/argocd sudo chmod +x
/tmp/argocd; sudo mv /tmp/argocd /usr/local/bin/ sudo apt-get install -y jq -qq } Write-Step "Verification finale" &
"$PSScriptRoot/01-install-tools.ps1" -Verify
```


04-postgresql.ps1

04

PostgreSQL dans Docker + Service ExternalName K8s

Lance le conteneur PostgreSQL 15 Alpine avec un volume persistant shipclass-pgdata. Attend que pg_isready réponde avant de créer les 5 bases de données. Détecte automatiquement l'hôte (host.docker.internal sur Windows/macOS, IP gateway Docker sur Linux). Crée le Service ExternalName dans K8s et le Secret avec les URLs de connexion.

Paramètre / Élément	Description
-Reset	Supprimer le conteneur et le volume (attention : perte des données)
pg_isready	Attente active du démarrage PostgreSQL avant création des BDD
host.docker.internal	Résolution automatique (Linux : IP gateway détectée)
pg-credentials Secret	Secret K8s avec les URLs PostgreSQL pour chaque microservice

Code source du script — 04-postgresql.ps1

■ 04-postgresql.ps1

```
#!/usr/bin/env pwsh # 04-postgresql.ps1 # Lance PostgreSQL dans Docker, cree les bases, configure le Service K8s
param([switch]$Reset) . "$PSScriptRoot/config.ps1" Write-Title "ETAPE 04 - PostgreSQL local (Docker)" if ($Reset) {
  docker stop $SC.PgContainer 2>$null; docker rm $SC.PgContainer 2>$null; docker volume rm shipclass-pgdata 2>$null;
  Write-OK "Conteneur supprime" } Write-Step "Demarrage PostgreSQL" $st = docker inspect $SC.PgContainer --format
"{{.State.Status}}" 2>$null if ($st -eq "running") { Write-Info "PostgreSQL deja en cours" } elseif ($st -eq "exited") {
  docker start $SC.PgContainer; Write-OK "PostgreSQL redemarre" } else { docker run -d --name $SC.PgContainer --restart
unless-stopped -e POSTGRES_PASSWORD=$SC.PgPassword -e POSTGRES_USER=$SC.PgUser -p "$($SC.PgPort):5432" -v
shipclass-pgdata:/var/lib/postgresql/data postgres:15-alpine Write-Info "Attente demarrage PostgreSQL..." $ok = $false
for ($i = 0; $i -lt 30; $i++) { $c = docker exec $SC.PgContainer pg_isready -U $SC.PgUser 2>&1 if ($c -match "accepting")
{ $ok = $true; break } Start-Sleep 2; Write-Host " ." -NoNewline } if (-not $ok) { Write-Fail "PostgreSQL n'a pas
demarre"; exit 1 } Write-OK "PostgreSQL pret" } Write-Step "Creation des bases de donnees" foreach ($db in
$SC.PgDatabases) { $ex = docker exec $SC.PgContainer psql -U $SC.PgUser -lqt 2>$null | Select-String "^\\s*$db\\s*$|" if
($ex) { Write-Info " '$db' existe deja" } else { docker exec $SC.PgContainer psql -U $SC.PgUser -c "CREATE DATABASE $db;"
2>$null Write-OK " '$db' creee" } } Write-Step "Service ExternalName K8s -> PostgreSQL" $dbHost = if ($IsLinux) { $gw =
docker inspect $SC.PgContainer --format "{{.NetworkSettings.Gateway}}" 2>$null if (-not $gw -or $gw -eq "") { $gw =
"172.17.0.1" } Write-Info "Linux: gateway Docker = $gw" $gw } else { "host.docker.internal" } Apply-Yaml @" apiVersion:
v1 kind: Service metadata: name: postgres-svc namespace: $($SC.Namespace) spec: type: ExternalName externalName: $dbHost
ports: - port: 5432 targetPort: 5432 "@ Write-Step "Secret K8s credentials BDD" kubectl delete secret pg-credentials -n
$SC.Namespace 2>$null $base =
"postgresql://$($SC.PgUser):$($SC.PgPassword)@postgres-svc.$($SC.Namespace).svc.cluster.local:5432" kubectl create secret
generic pg-credentials --from-literal=PG_USER=$SC.PgUser --from-literal=PG_PASSWORD=$SC.PgPassword --
from-literal=VESSEL_DB_URL="$base/vessels_db" --from-literal=CLASS_DB_URL="$base/classification_db" --
from-literal=CERT_DB_URL="$base/certificates_db" --from-literal=INSPECT_DB_URL="$base/inspections_db" --
namespace=$SC.Namespace Write-OK "=== PostgreSQL pret ! ===" Write-Info "Connexion directe : localhost:$($SC.PgPort) |
$($SC.PgUser) | $($SC.PgPassword)"
```


05-jfrog.ps1

05

JFrog Artifactory OSS dans Docker + dépôts + login

Lance JFrog Artifactory OSS (version gratuite) dans Docker. Attend ~90 secondes puis vérifie l'API /system/ping. Change automatiquement le mot de passe admin par défaut. Crée les dépôts docker-shipclass-local et helm-shipclass-local via l'API REST JFrog. Configure daemon.json Docker pour les registries insécures et crée l'imagePullSecret dans K8s.

Paramètre / Élément	Description
-Reset	Supprimer et recréer le conteneur JFrog
API REST JFrog	Création des dépôts et changement de mot de passe via Invoke-RestMethod
daemon.json	Ajout de localhost:8082 dans insecure-registries
jfrog-local-secret	Secret K8s imagePullSecret pour puller depuis JFrog

Code source du script — 05-jfrog.ps1

■ 05-jfrog.ps1

```
#!/usr/bin/env pwsh # 05-jfrog.ps1 # Lance JFrog Artifactory OSS dans Docker, configure les depots et le login Docker
param([switch]$Reset) . "$PSScriptRoot/config.ps1" Write-Title "ETAPE 05 - JFrog Artifactory OSS local" $JFROG_URL =
"http://localhost:$(($SC.JFrogPort))" if ($Reset) { docker stop shipclass-jfrog 2>$null; docker rm shipclass-jfrog 2>$null
} Write-Step "Demarrage JFrog Artifactory OSS" $st = docker inspect shipclass-jfrog --format "{{.State.Status}}" 2>$null
if ($st -eq "running") { Write-Info "JFrog deja en cours" } else { $jfrogData = Join-Path $SC.WorkDir "jfrog/var" New-Item
-ItemType Directory -Force $jfrogData | Out-Null docker run -d --name shipclass-jfrog --restart unless-stopped ` -p
"$(($SC.JFrogPort)):8082" -p "8081:8081" ` -v "${$jfrogData}:/var/opt/jfrog/artifactory" `
releases-docker.jfrog.io/jfrog/artifactory-oss:latest Write-Info "Attente demarrage JFrog (~90 secondes)..." Start-Sleep
90 $hdrs0 = @{ Authorization = "Basic " + [Convert]::ToBase64String([Text.Encoding]::UTF8.GetBytes("admin:password")) }
$ok = $false for ($i = 0; $i -lt 20; $i++) { try { $r = Invoke-RestMethod "$JFROG_URL/artifactory/api/system/ping"
-Headers $hdrs0 -EA Stop if ($r -eq "OK") { $ok = $true; break } } catch {} Start-Sleep 5; Write-Host " ." -NoNewline } if
(-not $ok) { Write-Fail "JFrog n'a pas démarre. Voir: docker logs shipclass-jfrog"; exit 1 } Write-OK "JFrog démarre" #
Changer le mot de passe $pwBody = @{ userName="admin"; oldPassword="password"; newPassword=$SC.JFrogPassword } |
ConvertTo-Json try { Invoke-RestMethod "$JFROG_URL/artifactory/api/security/users/authorization/changePassword" ` -Method
POST -Headers $hdrs0 -Body $pwBody -ContentType "application/json" Write-OK "Mot de passe admin change" } catch {
Write-Info "Mot de passe deja change" } } $hdrs = @{ Authorization = "Basic " +
[Convert]::ToBase64String([Text.Encoding]::UTF8.GetBytes("$($SC.JFrogUser): $($SC.JFrogPassword)")) "Content-Type" =
"application/json" } Write-Step "Creation des depots JFrog" @( @{ key=$SC.JFrogRepo; packageType="docker"; rclass="local"
}, @{ key="helm-shipclass-local"; packageType="helm"; rclass="local" } ) | ForEach-Object { try { Invoke-RestMethod
"$JFROG_URL/artifactory/api/repositories/$($_.key)" ` -Method PUT -Headers $hdrs -Body ($_ | ConvertTo-Json) Write-OK
"Depot '$($_.key)' cree" } catch { Write-Info "Depot '$($_.key)': $($_.Exception.Message)" } } Write-Step "Configuration
Docker insecure-registries" $dp = if ($IsWindows) { "$env:APPDATA\Docker\daemon.json" } ` elseif ($IsMacOS) {
"$HOME/.docker/daemon.json" } ` else { "/etc/docker/daemon.json" } $dc = if (Test-Path $dp) { Get-Content $dp -Raw |
ConvertFrom-Json } else { [PSCustomObject]@{} } if (-not $dc.'insecure-registries') { $dc | Add-Member -Name
"insecure-registries" -Value @("localhost:$(($SC.JFrogPort))" ) -MemberType NoteProperty $dc | ConvertTo-Json | Out-File $dp
-Encoding UTF8 Write-Info "daemon.json mis a jour. Redemarrer Docker Desktop." } docker login
"localhost:$(($SC.JFrogPort))" -u $SC.JFrogUser -p $SC.JFrogPassword Write-OK "Login Docker -> JFrog OK" kubectl delete
secret jfrog-local-secret -n $SC.Namespace 2>$null kubectl create secret docker-registry jfrog-local-secret `
--docker-server="localhost:$(($SC.JFrogPort))" ` --docker-username=$SC.JFrogUser ` --docker-password=$SC.JFrogPassword `
--namespace=$SC.Namespace Write-OK "Secret imagePullSecret cree" Write-OK "=== JFrog Artifactory pret ! ===
http://localhost:$(($SC.JFrogPort))"
```

06-build-images.ps1

06

Build et push de toutes les images Docker (avec Dockerfile de démo)

Itère sur tous les microservices (ou un seul avec -Service) et build/push les images. Si le Dockerfile est absent, génère automatiquement un Dockerfile Node.js minimal de démonstration qui expose un endpoint /health JSON. Supporte deux registries : kind (localhost:5001) ou jfrog (localhost:8082). Scan CVE optionnel avec Trivy.

Paramètre / Élément	Description
-Service all	Builder un seul service ou tous
-Tag latest v1.x.x	Tag de l'image Docker
-Registry kind jfrog	Choisir le registry cible
-ScanCVE	Activer le scan Trivy si disponible
Dockerfile auto-généré	Service de démo Node.js si pas de code source

Code source du script — 06-build-images.ps1

■ 06-build-images.ps1

```
#!/usr/bin/env pwsh # 06-build-images.ps1 # Build et push de toutes les images Docker des microservices ShipClass param([string]$Service = "all", [string]$Tag = "latest", [ValidateSet("kind","jfrog")] [string]$Registry = "kind", [switch]$ScanCVE ) . "$PSScriptRoot/config.ps1" Write-Title "ETAPE 06 - Build & Push des images ($Registry)" $reg = if ($Registry -eq "jfrog") { "localhost:$($SC.JFrogPort)/$($SC.JFrogRepo)" } else { $SC.Registry } Write-Info "Registry : $reg | Tag : $Tag" $services = if ($Service -eq "all") { $SC.Services } else { @($Service) } $root = Split-Path $PSScriptRoot -Parent $results = @() foreach ($svc in $services) { Write-Step "Build : $svc" $svcPath = if ($svc -eq "frontend") { Join-Path $root "frontend" } else { Join-Path $root "services" $svc } if (-not (Test-Path (Join-Path $svcPath "Dockerfile"))) { Write-Info "Dockerfile absent - creation d'un service de demo" New-Item -ItemType Directory -Force $svcPath | Out-Null $port = switch ($svc) { "vessel-service" { 3001 } "classification-service" { 3002 } "certificate-service" { 3003 } "inspection-service" { 3004 } "document-service" { 3005 } "notification-service" { 3006 } default { 3000 } } @" FROM node:20-alpine RUN addgroup -S app && adduser -S app -G app WORKDIR /app RUN printf '{"name":"$svc","version":"1.0.0","main":"index.js"}' > package.json RUN printf 'const http=require("http"),os=require("os");const s=http.createServer((q,r)=>{const h={"Content-Type":"application/json"};if(q.url==="/health"||q.url==="/ready"){r.writeHead(200,h);r.end(JSON.stringify({status:"ok",service:"$svc",version:"$Tag",host:os.hostname()}))}}else{r.writeHead(200,h);r.end(JSON.stringify({service:"$svc",message:"ShipClass local demo"}))}});s.listen($port,()=>console.log("$svc listening on :$port"));' > index.js USER app EXPOSE $port HEALTHCHECK --interval=30s --timeout=5s CMD wget -qO- http://localhost:$port/health||exit 1 CMD ["node","index.js"] "@ | Out-File "$svcPath/Dockerfile" -Encoding UTF8 } $imgTag = "$reg/$svc:$Tag" $imgLatest = "$reg/$svc:latest" docker build -t $imgTag -t $imgLatest $svcPath if ($LASTEXITCODE -ne 0) { $results += [PSCustomObject]@{ Service=$svc; Status="BUILD_FAIL" }; continue } if ($ScanCVE -and (Get-Command trivy -EA SilentlyContinue)) { Write-Info "Scan Trivy CVE..." trivy image --severity HIGH,CRITICAL --exit-code 0 $imgTag } docker push $imgTag if ($Tag -ne "latest") { docker push $imgLatest } $results += [PSCustomObject]@{ Service=$svc; Status="OK"; Image=$imgTag } Write-OK "$svc -> $imgTag" } Write-Title "Resume du build" $results | Format-Table -AutoSize $ko = $results | Where-Object { $_.Status -ne "OK" } if ($ko) { Write-Fail "$($ko.Count) echec(s)" } else { Write-OK "Tous les services buildes et pushes !" }
```

07-deploy-services.ps1

07

Déploiement K8s : ConfigMap, Secrets, Deployments, Services, Ingress

Crée le namespace shipclass, le ConfigMap global, le Secret applicatif, puis déploie chaque microservice avec Deployment (1 réplica, ressources requests/limits, liveness/readiness probes, securityContext), Service ClusterIP, et enfin l'Ingress global avec routing par path. Attend le rollout de chaque Deployment avant de passer au suivant.

Paramètre / Élément	Description
-Service all	Déployer un seul service ou tous
ConfigMap shipclass-config	Variables d'environnement non-sensibles
Secret shipclass-app-secrets	JWT, mot de passe BDD, clé AIS
Ingress multi-path	Routing /vessels, /classifications, /certificates, etc.
rollout status	Attente active du déploiement complet

Code source du script — 07-deploy-services.ps1

■ 07-deploy-services.ps1

```
#!/usr/bin/env pwsh # 07-deploy-services.ps1 # Deploie les microservices ShipClass sur le cluster kind param(
[string]$Service = "all", [ValidateSet("kind","jfrog")] [string]$Registry = "kind", [string]$Tag = "latest" ) .
"$PSScriptRoot/config.ps1" Write-Title "ETAPE 07 - Deploiement des microservices sur kind" $reg = if ($Registry -eq
"jfrog") { "localhost:$($SC.JFrogPort)/$($SC.JFrogRepo)" } else { $SC.Registry } $ns = $SC.Namespace kubectl create
namespace $ns 2>$null Write-Step "ConfigMap global" Apply-Yaml @" apiVersion: v1 kind: ConfigMap metadata: name:
shipclass-config namespace: $ns data: NODE_ENV: development LOG_LEVEL: debug POSTGRES_HOST:
postgres-svc.$ns.svc.cluster.local POSTGRES_PORT: "5432" POSTGRES_USER: $($SC.PgUser) JFROG_URL:
http://host.docker.internal:$($SC.JFrogPort)/artifactory "@ Write-Step "Secret applicatif" kubectl delete secret
shipclass-app-secrets -n $ns 2>$null kubectl create secret generic shipclass-app-secrets `
--from-literal=POSTGRES_PASSWORD=$SC.PgPassword ` --from-literal=JWT_SECRET="$(New-Guid)-local-dev" `
--from-literal=AIS_API_KEY="mock-ais-key-local" ` --namespace=$ns $svcDefs = @( @{ Name="vessel-service"; Port=3001;
SvcName="vessel-svc"; Path="/vessels" }, @{ Name="classification-service"; Port=3002; SvcName="classification-svc";
Path="/classifications" }, @{ Name="certificate-service"; Port=3003; SvcName="certificate-svc"; Path="/certificates" },
@{ Name="inspection-service"; Port=3004; SvcName="inspection-svc"; Path="/inspections" }, @{ Name="document-service";
Port=3005; SvcName="document-svc"; Path="/documents" }, @{ Name="notification-service"; Port=3006;
SvcName="notification-svc"; Path="/notifications" }, @{ Name="frontend"; Port=3000; SvcName="frontend-svc"; Path="/" } )
$selected = if ($Service -eq "all") { $svcDefs } else { $svcDefs | Where-Object { $_.Name -eq $Service } } foreach ($svc
in $selected) { Write-Step "Deploiement : $($svc.Name)" $img = "$reg/$($svc.Name):$Tag" Apply-Yaml @" apiVersion: apps/v1
kind: Deployment metadata: name: $($svc.Name) namespace: $ns labels: { app: $($svc.Name), version: "$Tag" } spec:
replicas: 1 selector: matchLabels: app: $($svc.Name) template: metadata: labels: { app: $($svc.Name), version: "$Tag" }
spec: imagePullSecrets: - name: jfrog-local-secret containers: - name: $($svc.Name) image: $img imagePullPolicy: Always
ports: - containerPort: $($svc.Port) envFrom: - configMapRef: { name: shipclass-config } - secretRef: { name:
shipclass-app-secrets } - secretRef: { name: pg-credentials } resources: requests: { cpu: 50m, memory: 128Mi } limits: {
cpu: 300m, memory: 256Mi } livenessProbe: httpGet: { path: /health, port: $($svc.Port) } initialDelaySeconds: 20
periodSeconds: 30 readinessProbe: httpGet: { path: /health, port: $($svc.Port) } initialDelaySeconds: 10 periodSeconds:
10 securityContext: runAsNonRoot: true runAsUser: 1000 allowPrivilegeEscalation: false --- apiVersion: v1 kind: Service
metadata: name: $($svc.SvcName) namespace: $ns spec: selector: { app: $($svc.Name) } ports: - port: 80 targetPort:
$($svc.Port) type: ClusterIP "@ Write-OK "$($svc.Name) -> $img" } Write-Step "Ingress global" $paths = ($selected |
Where-Object { $_.Path -ne "/" } | ForEach-Object { # ... (36 lignes supplémentaires dans le fichier .ps1) ...
```

08-argocd.ps1

08

ArgoCD via Helm + Ingress + Login CLI + Connexion repo Git

Installe ArgoCD en mode insecure (TLS géré par Ingress NGINX), expose via Ingress avec certificat mkcert. Récupère le mot de passe admin généré. Option -InstallGitea pour installer un serveur Git local complet (Gitea) dans kind. Si un GitRepo est fourni, connecte ArgoCD au repo et crée l'Application avec sync automated + self-heal + auto-prune.

Paramètre / Élément	Description
-GitRepo	URL du repo Git k8s-config
-GitToken	Personal Access Token GitHub/Gitea
-InstallGitea	Installer Gitea (Git local 100% offline)
--insecure	TLS géré par Ingress, pas par ArgoCD directement
sync-policy automated	Sync + self-heal + prune automatiques

Code source du script — 08-argocd.ps1

■ 08-argocd.ps1

```
#!/usr/bin/env pwsh # 08-argocd.ps1 # Installe ArgoCD, configure Ingress, connecte le repo Git param( [string]$GitRepo =
"", [string]$GitUser = "admin", [string]$GitToken = "", [switch]$InstallGitea ) . "$PSScriptRoot/config.ps1" Write-Title
"ETAPE 08 - ArgoCD GitOps sur kind" Write-Step "Installation ArgoCD" helm repo add argo
https://argoproj.github.io/argo-helm --force-update | Out-Null helm repo update | Out-Null helm upgrade --install argocd
argo/argo-cd `--namespace argocd --create-namespace `--set configs.params."server".insecure=true `--set
redis.resources.requests.memory=64Mi `--set server.resources.requests.memory=128Mi `--set
repoServer.resources.requests.memory=128Mi `--set applicationSet.resources.requests.memory=64Mi `--wait --timeout 5m
Write-OK "ArgoCD installe" Write-Step "Ingress ArgoCD" Apply-Yaml @" apiVersion: networking.k8s.io/v1 kind: Ingress
metadata: name: argocd-ingress namespace: argocd annotations: cert-manager.io/cluster-issuer: mkcert-issuer
nginx.ingress.kubernetes.io/ssl-redirect: "true" nginx.ingress.kubernetes.io/backend-protocol: "HTTP" spec:
ingressClassName: nginx tls: - hosts: [$(SC.Domains.ArgoCD)] secretName: argocd-tls rules: - host: $(SC.Domains.ArgoCD)
http: paths: - path: / pathType: Prefix backend: service: { name: argocd-server, port: { number: 80 } } "@ Write-Step
"Recuperation mot de passe admin ArgoCD" $enc = kubectl get secret argocd-initial-admin-secret -n argocd -o
jsonpath="{.data.password}" 2>$null $argoPw = if ($enc) {
[Text.Encoding]::UTF8.GetString([Convert]::FromBase64String($enc)) } else { "admin" } Write-OK "Mot de passe ArgoCD :
$argoPw" if ($InstallGitea) { Write-Step "Installation Gitea (serveur Git local)" helm repo add gitea-charts
https://dl.gitea.com/charts --force-update | Out-Null helm repo update | Out-Null helm upgrade --install gitea
gitea-charts/gitea `--namespace gitea --create-namespace `--set service.http.type=ClusterIP `--set
gitea.admin.username=admin `--set gitea.admin.password=gitea_local_pw `--set gitea.admin.email=admin@shipclass.local `
--set persistence.size=2Gi `--set resources.requests.memory=128Mi `--wait --timeout 4m Apply-Yaml @" apiVersion:
networking.k8s.io/v1 kind: Ingress metadata: name: gitea-ingress namespace: gitea annotations:
cert-manager.io/cluster-issuer: mkcert-issuer spec: ingressClassName: nginx tls: - hosts: [$(SC.Domains.Gitea)]
secretName: gitea-tls rules: - host: $(SC.Domains.Gitea) http: paths: - path: / pathType: Prefix backend: service: {
name: gitea-http, port: { number: 3000 } } "@ Write-OK "Gitea : https://$(SC.Domains.Gitea) (admin/gitea_local_pw)"
$GitRepo = "https://$(SC.Domains.Gitea)/admin/k8s-config" $GitToken = "gitea_local_pw" } Write-Step "Login ArgoCD CLI"
Start-Sleep 8 argocd login $SC.Domains.ArgoCD --username admin --password $argoPw --insecure --grpc-web if ($GitRepo) {
Write-Step "Connexion repo Git : $GitRepo" argocd repo add $GitRepo --username $GitUser --password $GitToken
--insecure-skip-server-verification Write-Step "Creation Application ArgoCD shipclass-local" argocd app create
shipclass-local `--repo $GitRepo `--path "apps/shipclass/overlays/local" `--dest-server https://kubernetes.default.svc
`--dest-namespace $SC.Namespace `--sync-policy automated `--auto-prune --self-heal argocd app sync shipclass-local
argocd app list } Write-OK "=== ArgoCD pret ! ===" Write-Info "UI : https://$(SC.Domains.ArgoCD)" # ... (2 lignes
supplémentaires dans le fichier .ps1) ...
```

09-monitoring.ps1

09

Prometheus + Grafana + AlertManager + ServiceMonitor ShipClass

Installe kube-prometheus-stack avec rétention 1 jour (suffisant en local), Grafana avec mot de passe, Alertmanager désactivé (bruit en local). Expose Grafana via Ingress HTTPS avec certificat mkcert. Crée un ServiceMonitor pour les microservices ShipClass. Affiche les IDs des dashboards Grafana à importer manuellement.

Paramètre / Élément	Description
kube-prometheus-stack	Stack complète : Prometheus + Grafana + Node Exporter + kube-state-metrics
retention=1d	1 jour suffit pour le dev local
alertmanager.enabled=false	Moins de bruit en local
ServiceMonitor	Scraping automatique des métriques des pods ShipClass
Dashboards IDs	15760, 13770, 6417 à importer dans l'UI Grafana

Code source du script — 09-monitoring.ps1

■ 09-monitoring.ps1

```
#!/usr/bin/env pwsh # 09-monitoring.ps1 # Installe Prometheus + Grafana sur kind . "$PSScriptRoot/config.ps1" Write-Title
"ETAPE 09 - Prometheus + Grafana" Write-Step "Installation kube-prometheus-stack" helm repo add prometheus-community
https://prometheus-community.github.io/helm-charts --force-update | Out-Null helm repo update | Out-Null helm upgrade
--install monitoring prometheus-community/kube-prometheus-stack ` --namespace monitoring --create-namespace ` --set
prometheus.prometheusSpec.retention=1d ` --set prometheus.prometheusSpec.retentionSize="3GB" ` --set
prometheus.prometheusSpec.resources.requests.memory=256Mi ` --set prometheus.prometheusSpec.resources.requests.cpu=100m `
--set grafana.adminPassword=grafana_local_pw ` --set grafana.resources.requests.memory=128Mi ` --set
grafana.resources.requests.cpu=50m ` --set alertmanager.enabled=false ` --set "grafana.ingress.enabled=true" ` --set
"grafana.ingress.ingressClassName=nginx" ` --set
"grafana.ingress.annotations.cert-manager.io/cluster-issuer=mkcert-issuer" ` --set
"grafana.ingress.hosts[0]=$($SC.Domains.Grafana)" ` --set "grafana.ingress.tls[0].secretName=grafana-tls" ` --set
"grafana.ingress.tls[0].hosts[0]=$($SC.Domains.Grafana)" ` --wait --timeout 6m Write-OK "Prometheus + Grafana installes"
Write-Step "ServiceMonitor pour ShipClass" Apply-Yaml @" apiVersion: monitoring.coreos.com/v1 kind: ServiceMonitor
metadata: name: shipclass-monitor namespace: monitoring labels: release: monitoring spec: selector: matchLabels: app:
vessel-service namespaceSelector: matchNames: [$($SC.Namespace)] endpoints: - port: http path: /metrics interval: 30s "@
Write-OK "=== Monitoring pret ! ===" Write-Info "Grafana : https://$($SC.Domains.Grafana) (admin/grafana_local_pw)"
Write-Info "Dashboards recommandes (import via UI) :" Write-Info " ID 15760 - Node Exporter Full" Write-Info " ID 13770 -
Kubernetes Cluster" Write-Info " ID 6417 - Kubernetes Pods"
```

10-daily.ps1

10

Commandes quotidiennes : start, stop, status, rebuild, db, logs

Script principal pour le travail au quotidien. L'action 'start' démarre les conteneurs Docker et vérifie le cluster. 'status' affiche l'état de tout l'environnement en un coup d'oeil. 'rebuild' relance le build et fait un rolling restart. 'db' ouvre une session psql interactive. 'logs' streames les logs d'un service. 'port-forward' expose Prometheus en local.

Paramètre / Élément	Description
-Action start	Démarrer Docker + vérifier cluster kind
-Action status	Vue d'ensemble : Docker, pods, certs, ArgoCD
-Action rebuild -Service	Rebuild + rolling restart d'un service
-Action db	Session psql interactive sur vessels_db
-Action logs -Service	Streaming des logs kubectl
-Action port-forward	Expose Prometheus sur localhost:9090

Code source du script — 10-daily.ps1

■ 10-daily.ps1

```
#!/usr/bin/env pwsh # 10-daily.ps1 # Commandes quotidiennes : start, stop, status, rebuild, db, logs, reset-all param(
[ValidateSet("start","stop","status","rebuild","db","logs","reset-all","port-forward")] [string]$Action = "status",
[string]$Service = "all", [string]$Tag = "latest" ) . "$PSScriptRoot/config.ps1" Write-Title "DAILY - Action : $Action"
switch ($Action) { "start" { Write-Step "Demarrage de l'environnement" docker info *>>$null if ($LASTEXITCODE -ne 0) {
Write-Fail "Docker non lance !"; exit 1 } @($SC.PgContainer, "shipclass-jfrog") | ForEach-Object { $st = docker inspect
$_ --format "{{.State.Status}}" 2>$null if ($st -eq "exited") { docker start $_; Write-OK "$_ démarre" } elseif ($st -eq
"running") { Write-Info "$_ deja en cours" } else { Write-Info "$_ non cree" } } kubectl config use-context $SC.Context
2>$null $nodes = kubectl get nodes --no-headers 2>$null if ($nodes -match "Ready") { Write-OK "Cluster kind operationnel"
} Write-OK "URLs disponibles : " Write-Info " API : https://$($SC.Domains.Api)/vessels/health" Write-Info " ArgoCD :
https://$($SC.Domains.ArgoCD)" Write-Info " Grafana : https://$($SC.Domains.Grafana)" Write-Info " JFrog :
http://localhost:$($SC.JFrogPort)" } "stop" { docker stop $SC.PgContainer 2>$null; Write-OK "PostgreSQL arrete" docker
stop shipclass-jfrog 2>$null; Write-OK "JFrog arrete" Write-Info "Cluster kind conserve en memoire (redemarrage
automatique avec Docker)" } "status" { Write-Info "--- Docker ---" @($SC.PgContainer, "shipclass-jfrog",
$SC.RegistryName) | ForEach-Object { $st = docker inspect $_ --format "{{.State.Status}}" 2>$null $ico = if ($st -eq
"running") { "[OK]" } else { "[KO]" } Write-Host " $ico $_ : $(if ($st) { $st } else { 'non cree' })" } Write-Info "`n---
Pods ShipClass ---" kubectl get pods -n $SC.Namespace --no-headers 2>$null Write-Info "`n--- Certificats TLS ---" kubectl
get certificates -A --no-headers 2>$null Write-Info "`n--- ArgoCD Apps ---" argocd app list --grpc-web 2>$null }
"rebuild" { & "$PSScriptRoot/06-build-images.ps1" -Service $Service -Tag $Tag if ($LASTEXITCODE -eq 0) { if ($Service -eq
"all") { $SC.Services | ForEach-Object { kubectl rollout restart deployment/$_ -n $SC.Namespace 2>$null } } else {
kubectl rollout restart deployment/$Service -n $SC.Namespace kubectl rollout status deployment/$Service -n $SC.Namespace
--timeout=90s } } } "db" { Write-Info "Session psql sur vessels_db (Ctrl+D ou \q pour quitter)" docker exec -it
$SC.PgContainer psql -U $SC.PgUser -d vessels_db } "logs" { if ($Service -eq "all") { Write-Info "Specifiez -Service
<nom>"; return } kubectl logs -f deployment/$Service -n $SC.Namespace --tail=100 } "port-forward" { Write-Info
"Port-forwards disponibles : " Write-Info " PostgreSQL : kubectl port-forward svc/postgres-svc 5432:5432 -n
$($SC.Namespace)" Write-Info " Prometheus : kubectl port-forward svc/monitoring-kube-prometheus-prometheus 9090:9090 -n
monitoring" Write-Info " ArgoCD API : kubectl port-forward svc/argocd-server 8443:443 -n argocd" Write-Info "En lancer un
: kubectl port-forward svc/monitoring-kube-prometheus-prometheus 9090:9090 -n monitoring" kubectl port-forward
svc/monitoring-kube-prometheus-prometheus 9090:9090 -n monitoring } "reset-all" { Write-Info "ATTENTION : toutes les
donnees seront supprimees !" $confirm = Read-Host "Confirmer ? (oui/non)" if ($confirm -ne "oui") { Write-Info "Annule";
return } kind delete cluster --name $SC.ClusterName 2>$null docker stop $SC.PgContainer shipclass-jfrog $SC.RegistryName
2>$null docker rm $SC.PgContainer shipclass-jfrog $SC.RegistryName 2>$null docker volume rm shipclass-pgdata 2>$null
Write-OK "Reset complet. Relancer 02-create-cluster.ps1 pour recommencer." } }
```

11-verify.ps1

11

Tests de bout-en-bout : Docker, K8s, HTTPS, BDD, API

Lance une batterie de tests automatisés : disponibilité Docker, état des conteneurs, nœuds K8s Ready, pods Running, certificats TLS READY, HTTPS API/ArgoCD/Grafana, connexion PostgreSQL, et enfin un test fonctionnel complet de création d'un navire via l'API REST avec validation de la réponse.

Paramètre / Élément	Description
TC function	Fonction de test avec compteur d'erreurs
Tests HTTP	Invoke-WebRequest avec timeout sur tous les endpoints HTTPS
Test PostgreSQL	Connexion docker exec psql et vérification de la BDD
Test API fonctionnel	POST /vessels et vérification de la réponse JSON
Résumé final	Compte des tests OK/KO avec conseils de debug

Code source du script — 11-verify.ps1

■ 11-verify.ps1

```
#!/usr/bin/env pwsh # 11-verify.ps1 # Tests de bout-en-bout pour valider que tout fonctionne . "$PSScriptRoot/config.ps1"
Write-Title "ETAPE 11 - Verification complete de l'environnement" $errors = 0 function TC { param($Name, $Block) try { $r
= & $Block if ($r) { Write-OK $Name } else { Write-Fail $Name; $script:errors++ } } catch { Write-Fail "$Name :
$($_.Exception.Message)"; $script:errors++ } } Write-Step "Docker" TC "Docker operationnel" { docker info *>$null;
$LASTEXITCODE -eq 0 } TC "PostgreSQL running" { (docker inspect $SC.PgContainer --format "{{.State.Status}}") 2>$null -eq
"running" } TC "Kind Registry running" { (docker inspect $SC.RegistryName --format "{{.State.Status}}") 2>$null -eq
"running" } Write-Step "Kubernetes" TC "Cluster accessible" { kubectl cluster-info 2>$null; $LASTEXITCODE -eq 0 } TC
"Noeuds Ready (min 2)" { $n = kubectl get nodes --no-headers 2>$null ($n -split "`n" | Where-Object { $_ -match " Ready "
}).Count -ge 2 } Write-Step "Pods ShipClass" $pods = kubectl get pods -n $SC.Namespace --no-headers 2>$null if ($pods) {
($pods -split "`n") | Where-Object { $_ -ne "" } | ForEach-Object { $c = $_ -split '\s+' TC "Pod $($c[0])" { $c[2] -eq
"Running" } } } else { Write-Info "Aucun pod dans $($SC.Namespace)" } Write-Step "Certificats TLS" $certs = kubectl get
certificates -A --no-headers 2>$null if ($certs) { ($certs -split "`n") | Where-Object { $_ -ne "" } | ForEach-Object { $c
= $_ -split '\s+' TC "Certificat $($c[1]) READY" { $c[2] -eq "True" } } } Write-Step "API HTTPS" TC
"https://$($SC.Domains.Api)/vessels/health" { try { (Invoke-WebRequest "https://$($SC.Domains.Api)/vessels/health"
-UseBasicParsing -TimeoutSec 5 -EA Stop).StatusCode -eq 200 } catch { $false } } TC "ArgoCD UI accessible" { try {
(Invoke-WebRequest "https://$($SC.Domains.ArgoCD)" -UseBasicParsing -TimeoutSec 5 -EA Stop).StatusCode -in @(200,307) }
catch { $false } } TC "Grafana UI accessible" { try { (Invoke-WebRequest "https://$($SC.Domains.Grafana)"
-UseBasicParsing -TimeoutSec 5 -EA Stop).StatusCode -eq 200 } catch { $false } } Write-Step "PostgreSQL" TC "vessels_db
accessible" { $r = docker exec $SC.PgContainer psql -U $SC.PgUser -d vessels_db -c "\l" -q 2>&1 $r -match "vessels_db" }
Write-Step "Test fonctionnel : creation d'un navire" try { $body = @{ imo_number="IM09999$(Get-Random -Max 999)";
name="MV Test Local"; flag="FR"; type="Bulk Carrier"; grt=45000 } | ConvertTo-Json $r = Invoke-RestMethod
"https://$($SC.Domains.Api)/vessels" -Method POST -ContentType "application/json" -Body $body -EA Stop Write-OK "Navire
cree : $($r.name) (ID: $($r.id))" } catch { Write-Info "Test creation navire : $($_.Exception.Message)" } Write-Title
"Resume" if ($errors -eq 0) { Write-OK "TOUS LES TESTS REUSSIS - Environnement 100% operationnel !" } else { Write-Fail
"$errors test(s) en echec" Write-Info "Debug : ./10-daily.ps1 -Action status" Write-Info "Logs : ./10-daily.ps1 -Action
logs -Service <nom>" }
```


00-bootstrap.ps1

00

Bootstrap complet : installe tout dans l'ordre en une seule commande

Orchestre l'exécution séquentielle de tous les scripts dans le bon ordre. Affiche la progression et la durée. En cas d'erreur sur une étape, propose de continuer ou d'arrêter. Les paramètres -SkipTools et -SkipJFrog permettent d'éviter les étapes déjà faites. Affiche un récapitulatif des URLs et commandes à la fin.

Paramètre / Élément	Description
-SkipTools	Ignorer l'installation des outils (déjà installés)
-SkipJFrog	Utiliser uniquement le registry kind sans JFrog
-GitRepo / -GitToken	Passer l'URL Git pour configurer ArgoCD
Continue on error	Proposer de continuer si une étape échoue
Récapitulatif URLs	Affichage final de toutes les URLs et commandes

Code source du script — 00-bootstrap.ps1

■ 00-bootstrap.ps1

```
#!/usr/bin/env pwsh # 00-bootstrap.ps1 # Lance l'installation complete de l'environnement ShipClass local dans l'ordre #
Usage : ./00-bootstrap.ps1 # ./00-bootstrap.ps1 -SkipTools (si deja installes) # ./00-bootstrap.ps1 -GitRepo <url>
-GitToken <token> param( [switch]$SkipTools, [switch]$SkipJFrog, [string]$GitRepo = "", [string]$GitToken = "" )
$ErrorActionPreference = "Continue" Write-Host @" +=====+ |
ShipClass Local - Bootstrap Complet | | Kubernetes (kind) + ArgoCD + JFrog + cert-manager | | PostgreSQL + Prometheus +
Grafana + Ingress NGINX | +=====+ "@ -ForegroundColor Cyan $start
= Get-Date $steps = @( @{ Script="01-install-tools.ps1"; Skip=$SkipTools; Label="Installation des outils" }, @{
Script="02-create-cluster.ps1"; Skip=$false; Label="Cluster kind + Registry" }, @{ Script="03-tls-ingress.ps1";
Skip=$false; Label="TLS mkcert + Ingress NGINX" }, @{ Script="04-postgresql.ps1"; Skip=$false; Label="PostgreSQL local"
}, @{ Script="05-jfrog.ps1"; Skip=$SkipJFrog; Label="JFrog Artifactory OSS" }, @{ Script="06-build-images.ps1";
Skip=$false; Label="Build des images Docker" }, @{ Script="07-deploy-services.ps1"; Skip=$false; Label="Deploiemet
microservices" }, @{ Script="08-argocd.ps1"; Skip=$false; Label="ArgoCD GitOps" }, @{ Script="09-monitoring.ps1";
Skip=$false; Label="Prometheus + Grafana" }, @{ Script="11-verify.ps1"; Skip=$false; Label="Verification finale" } ) $i =
0; $ok = 0; $ko = 0 foreach ($step in $steps) { $i++ if ($step.Skip) { Write-Host " [SKIP] [$i/$(($steps.Count))
] $($step.Label)" -ForegroundColor DarkGray continue } Write-Host "`n [ >>> ] [$i/$(($steps.Count))] $($step.Label)..."
-ForegroundColor Yellow $sp = Join-Path $PSScriptRoot $step.Script if (-not (Test-Path $sp)) { Write-Host " [SKIP] Script
$(($step.Script)) introuvable" -ForegroundColor DarkGray; continue } try { if ($step.Script -eq "08-argocd.ps1" -and
$GitRepo) { & $sp -GitRepo $GitRepo -GitToken $GitToken } else { & $sp } Write-Host " [OK] $($step.Label)"
-ForegroundColor Green; $ok++ } catch { Write-Host " [ERR] $($step.Label) : $(($_.Exception.Message))" -ForegroundColor
Red; $ko++ $ans = Read-Host " Continuer quand meme ? (o/n)" if ($ans -ne "o") { break } } } $dur =
[math]::Round(((Get-Date) - $start).TotalMinutes, 1) Write-Host @"
+=====+ | Termine en $dur minutes ($ok OK, $ko erreur(s)) |
+-----+ | API https://api.shipclass.local | | App
https://shipclass.local | | ArgoCD https://argocd.shipclass.local | | Grafana https://grafana.shipclass.local | | JFrog
http://localhost:8082 (admin/ShipLocal2024!) | | PostgreSQL localhost:5432 (pgadmin/shipclass_local_pw) |
+-----+ | Commandes quotidiennes : | | ./10-daily.ps1 -Action
start | | ./10-daily.ps1 -Action status | | ./10-daily.ps1 -Action rebuild -Service vessel-service | | ./10-daily.ps1
-Action db | | ./10-daily.ps1 -Action logs -Service vessel-service |
+=====+ "@ -ForegroundColor Cyan
```


PARTIE 3 — Aide-Mémoire & URLs

Récapitulatif de toutes les URLs locales, mots de passe, commandes fréquentes et guide de dépannage rapide.

URLs et Accès Locaux

Service	URL	Login	Script qui le crée
API ShipClass	https://api.shipclass.local	Token JWT	07-deploy-services.ps1
Frontend	https://shipclass.local	—	07-deploy-services.ps1
ArgoCD	https://argocd.shipclass.local	admin / voir secret	08-argocd.ps1
Grafana	https://grafana.shipclass.local	admin / grafana_local_pw	09-monitoring.ps1
JFrog Artifactory	http://localhost:8082	admin / ShipLocal2024!	05-jfrog.ps1
PostgreSQL	localhost:5432	pgadmin / shipclass_local_pw	04-postgresql.ps1
Registry kind	http://localhost:5001	— (sans auth)	02-create-cluster.ps1
Prometheus	http://localhost:9090	— (port-forward)	10-daily.ps1 -Action port-forward
Gitea (optionnel)	https://gitea.shipclass.local	admin / gitea_local_pw	08-argocd.ps1 -InstallGitea

Commandes quotidiennes les plus fréquentes

```
# Démarrer l'environnement le matin pwsh ./10-daily.ps1 -Action start # Voir l'état de tout l'environnement pwsh ./10-daily.ps1 -Action status # Rebuild un service après modification du code pwsh ./10-daily.ps1 -Action rebuild -Service vessel-service # Rebuild TOUS les services pwsh ./10-daily.ps1 -Action rebuild -Service all # Accéder à la base de données interactivement pwsh ./10-daily.ps1 -Action db # Voir les logs en temps réel pwsh ./10-daily.ps1 -Action logs -Service vessel-service # Arrêter l'environnement le soir pwsh ./10-daily.ps1 -Action stop # Lancer les tests de bout-en-bout pwsh ./11-verify.ps1 # Forcer un sync ArgoCD argocd app sync shipclass-local --grpc-web # Voir tous les pods de tous les namespaces kubectl get pods -A # Voir les certificats TLS kubectl get certificates -A # Redémarrer un service sans rebuild kubectl rollout restart deployment/vessel-service -n shipclass # Accès direct à la BDD depuis le terminal (sans psql installé localement) docker exec -it shipclass-postgres psql -U pgadmin -d vessels_db
```

Dépannage rapide

Symptôme	Cause probable	Solution PS1
Pod en CrashLoopBackOff	Image manquante ou variable d'env incorrecte	<pre>kubectl describe pod <pod> -n shipclass kubectl logs <pod> -n shipclass --previous</pre>
Certificate READY=False	ClusterIssuer non prêt après reset	<pre>kubectl rollout restart deploy/cert-manager -n cert-manager Start-Sleep 30; kubectl get certificates -A</pre>
Ingress 502 Bad Gateway	Le pod backend n'est pas Running	<pre>pwsh ./10-daily.ps1 -Action status pwsh ./10-daily.ps1 -Action rebuild -Service <nom></pre>
JFrog ne répond pas	Démarrage lent (~90s) ou arrêté	<pre>docker logs shipclass-jfrog --tail 20 docker start shipclass-jfrog</pre>
host.docker.internal Linux	Non résolu nativement sur Linux	<pre># Script 04 détecte automatiquement l'IP gateway # Vérifier avec : docker inspect shipclass-postgres --format '{{.NetworkSettings.Gateway}}'</pre>
ArgoCD OutOfSync	Commit dans Git non encore synchronisé	<pre>argocd app sync shipclass-local --grpc-web --force</pre>
Cluster kind non réactif	Docker Desktop redémarré, kind en recovery	<pre>kubectl config use-context kind-shipclass-local kubectl get nodes # attendre ~30s</pre>

■ Environnement 100% local, 0 € de coût

L'intégralité du projet ShipClass tourne sur votre machine sans aucun compte cloud. Temps de setup depuis zéro (avec connexion internet correcte) : ~30-45 minutes. Temps de redémarrage quotidien : ~2 minutes (./10-daily.ps1 -Action start). Tous les scripts sont idempotents : vous pouvez les relancer en cas d'erreur.